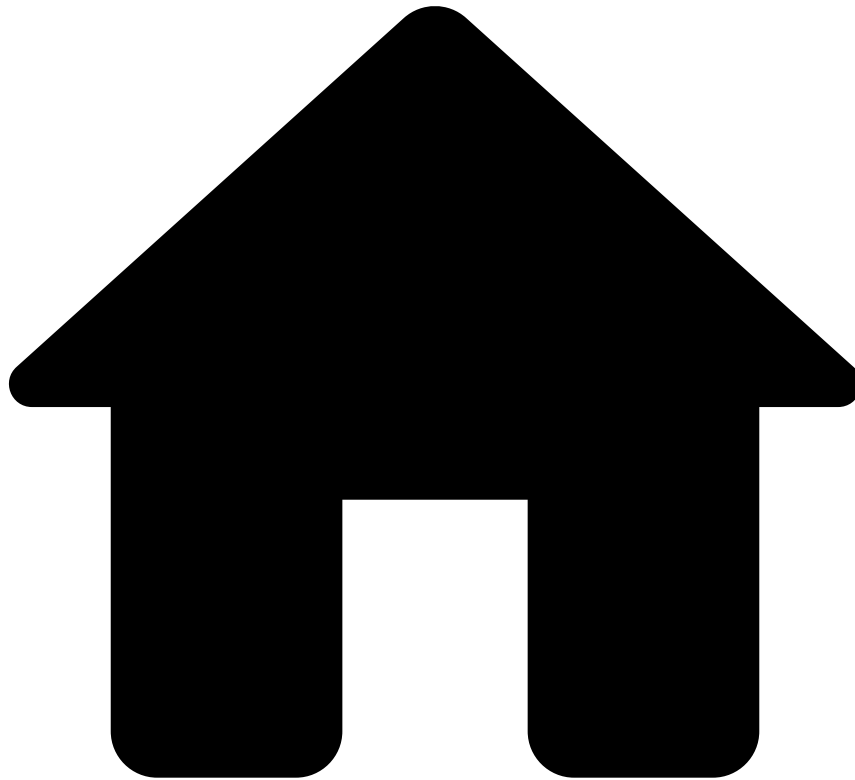


•



- [Working with Applications](#)
- Automatic Application Repair

On this page

Automatic Application Repair

Was this page helpful? 

Pulse has an **Auto-Repair** functionality that monitors the state of the cluster and decides whether it needs to be repaired or not. If the application is in an invalid state, Pulse will automatically trigger repair actions on the respective components. Fully repairing the cluster might involve executing several fix actions in sequence.

This functionality is not designed to fix the entire application all at once. It rather focuses on maintaining a small number of state validation rules, which when triggered and composed, allow the overall system to converge to a stable state.

It is possible that the **Auto-Repair** triggers a fix to a problem that will not return the system to a stable state. However, it guarantees that the system will be one step closer to a stable state. When each step is finished, it will allow for more fixes to be issued until the stable state is reached.

Fixing issues🔗

Pulse **Auto-Repair** consists in actions that aim at fixing the following issues:

- **Under replication:** Pulse will ensure that there are at least as many App Engines as the configured replication factor. If an AppEngine crashes, Pulse will automatically launch a new AppEngine node until reaching the replication factor.
- **Over replication:** Pulse will ensure that the number of AppEngines for a provided application does not exceed the configured replication factor. If Pulse detects that there are more nodes running the application than the number defined in the replication factor, then Pulse will shut down the application in as many nodes as required to adhere to the replication factor. Selecting which nodes to shut down is not deterministic.
- **Inconsistent versions:** When different nodes are running different versions of the application, or are not running the expected version, Pulse will automatically try to publish the expected version. In most cases this is the latest stable published version. If the application is in a *bootstrapped* state, then Pulse ensures that all AppEngines are in that state.

Configuring Auto-Repair🔗

To enable or disable the **Auto-Repair**, execute the following actions:

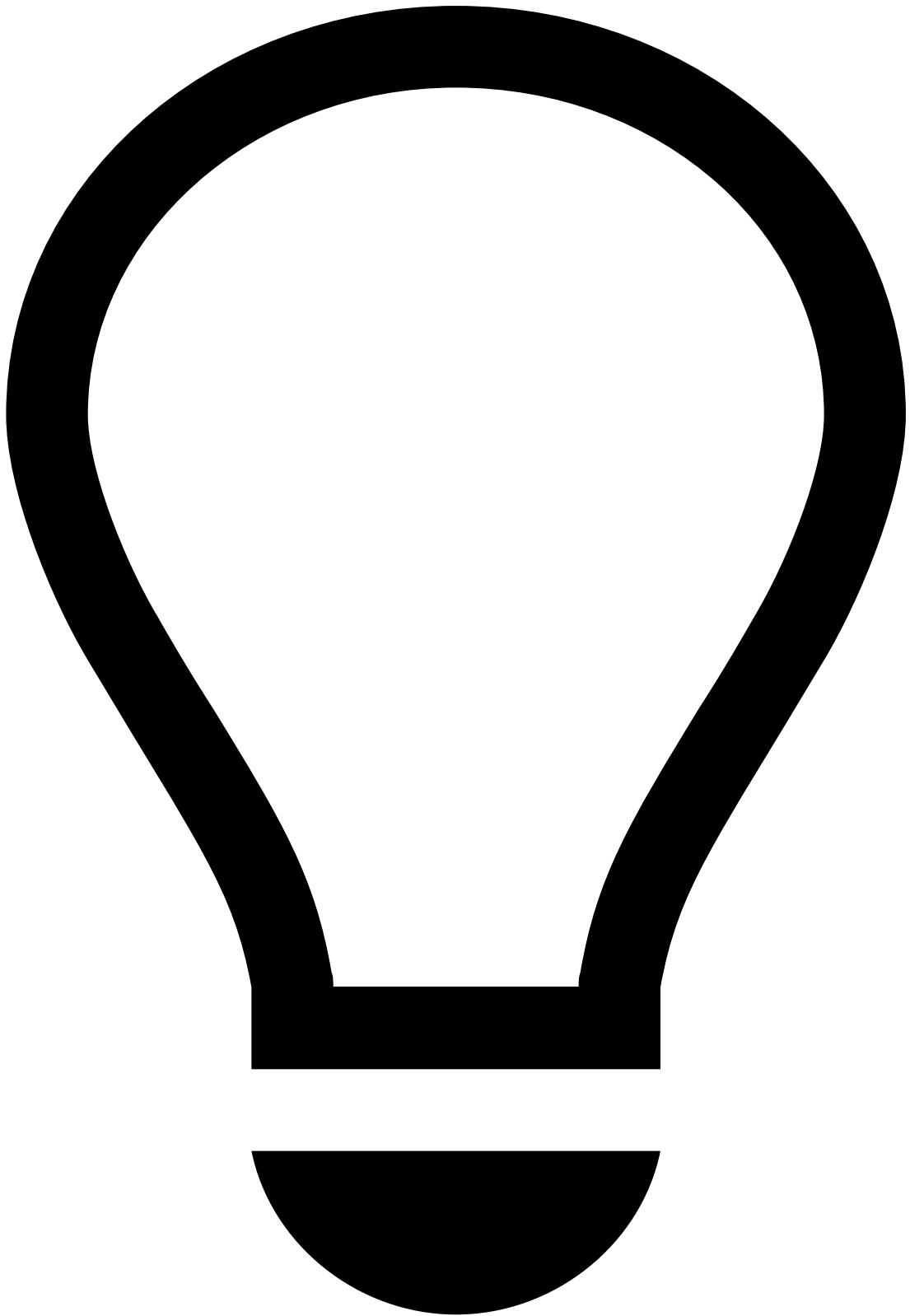
1. In your Pulse application, use the **Options** menu and select the **Edit** option.
2. Toggle the **Auto-Repair** configuration accordingly:

When the **Auto-Repair** feature is taking action, Pulse informs you that it is fixing your application.

In addition, Pulse also includes configuration properties that allow you to define the behavior of the **Auto-Repair** functionality individually by each faulty detection rule.

To configure the **Auto-Repair** functionality, execute the following actions in the Pulse advanced configuration:

1. Activate or deactivate each type of issues to be detected and fixed. For example, if you set the `fixInconsistentVersions` property to `false`, whenever Pulse detects you are trying to publish an application and there are different versions running, you must synchronize them manually.
2. Set the `checkPeriodMillis` property to, for example, 2 minutes.
3. Set the `latencyActuationCycles` property to, for example, 3 checks. Pulse only resets the cycle count when the entire cluster is stable, and not when a fix action is triggered.



Recommended configuration

The recommended configuration is that you set:

- `checkPeriodMillis` to a small interval.
- `latencyActuationCycles` to approximately 3 cycles.

This ensures some balance as fixes are quickly issued, and Pulse still allows for some space when waiting for momentary problems (for example, network partition) to fix on their own. In this example, it would always take minimum 6 minutes to issue a fix.